Gate detection — what works & what is left to test

Mahbod Tajdini
mtajdini@ethrobotics.ch — ETHRC Aeronautics

2026-05-10

TL;DR

The detection pipeline (segmentation + multi-task keypoint head + classical QuAdGate) trains, exports to TensorRT, and produces sub-pixel-accurate corners on TII piloted footage. The deployment-headline checkpoint `best_deeplabv3plus_keypoint.pth` achieves **85.1 % detection rate** at **1.01 px median** corner error on flight-01p-ellipse, single forward pass, no prior, no EKF state, 102 fps wall-clock on an RTX 5090 (0.47 ms TRT FP16). What is *not* yet measured: detection rate on flights other than the one we keep benching, false-positive rate on no-gate frames, real-recorded footage (blocked on Drive data).

1 What works today

Numbers are on `flight-01p-ellipse` (TII v3.0.0 piloted, 423 frames, seed 42), reproducible from report 07.

Segmentation backbone.
DeepLabV3+ with ResNet-34 + ImageNet pretrain — val IoU **0.872**. *report 05 / 07*

Keypoint head.
Multi-task 4-channel Gaussian-heatmap head, sub-pixel decode via soft-argmax. *report 07*

Deployment-mode bench (`model_direct`).
85.1 % detection / **1.01 px median** / 63.7 % within 5 px on flight-01p-ellipse (no GT prior, no EKF state). *report 07*

Prior-mode framework.
Five priors wired in one bench script: `gt`, `noisy`, `none`, `model`, `model_direct`, `ekf`. *reports 06 / 07 / 08*

TRT FP16 export.
Multi-task graph exports to ONNX (opset 18) and builds an FP16 TRT engine in ~ 6 s. **0.47 ms / 2128 fps on RTX 5090**, projects ∼50–100 fps on Jetson Orin NX. *report 03 + `bench_trt_keypoint.py`*

TRT numerical correctness.
TRT FP16 output matches torch FP32 to $|trt - torch|_\infty < 5 \times 10^{-4}$ on uniform-$[0, 1]$ input. *same*

Multi-gate sibling checkpoint.
Multi-gate Gaussian targets + coherent-quad decode at inference; 88.7 % det / 1.26 px median (tied with report 07 on within-5 px). *report 10*

Knock-on consequences ruled in. TII-distribution detection is solved (within the constraints of the report-07 metric); architecture exports cleanly to TRT; multi-task did not regress seg val IoU; the multi-gate-target experiment closed without moving the headline either way.

2 What is *not* measured (and matters for detection reliability)

#	Open test	Cost
1	Cross-flight bench: pick 4–5 flights from `*-ellipse`, `*-lemniscate`, `*-trackRATM`. Tells us if 85 % generalises across camera trajectory and track layout.	30 min
2	False-positive rate on no-gate frames: scan the dataset for empty label files (start of each flight, between-gate transits), measure how often the model fires anyway at the current 0.3 threshold.	30 min
3	Cross-flight bench of the multi-gate sibling ckpt: does the +3.6 pp detection-rate gain in report 10 hold out of distribution?	15 min
4	Confidence calibration: replace $peak \geq 0.3$ with peak + spatial concentration (KL divergence of the peak's 7×7 window from a $\sigma=3$ Gaussian). Catches doorway-style false positives in report 07's sample panel.	2 h
5	Bench on `flight-01p-fixed` (the original 82 % headline): blocked on Drive data + missing `scripts/prepare_labels.py`.	blocked
6	Real-footage bench (the README's 39 % number): blocked on Drive data.	blocked
7	Heavier motion-blur augmentation + retrain. Current aug caps at 15 px blur kernel; trackRATM samples wander on fast-turn frames.	1 day
8	Real-data fine-tuning (encoder frozen, decoder + head on team-recorded footage).	blocked

What the open tests do *not* cover. "Which gate is the next one to fly through" is not a perception question under the current architecture — it is a planner-side concern that needs an EKF + track-plan + plane-test (the MonoRace recipe, Bahnam et al. 2026). That workstream is items 4–6 on the master rank list (Kalman fusion, ROS2, Orin NX) and intentionally outside the scope of "are gates detected reliably."

3 Recommended order

Items 1 and 2 cost about an hour combined and are pure measurement. They tell us whether items 4 / 7 are actually worth doing, or whether the current numbers already generalise. Recommended sequence:

1. **Item 1 (cross-flight bench)** first — tells us whether 85.1 % is representative or one-flight luck. 30 min.
2. **Item 2 (false-positive rate)** — closes the precision picture. If FP rate is already <2 %, item 4 isn't worth doing. 30 min.
3. **Item 4 (confidence calibration)** only if item 2 shows a real false-positive problem. 2 h.
4. **Item 3 (multi-gate sibling cross-flight)** — informational, can be skipped without affecting the deployment path.

Items 5–6 unblock once the Drive data lands on qolam; item 7 is the only remaining unblocked retrain lever and only justifies its cost if item 1 shows trajectory-specific failures.

Bottom line. The model detects gates well on the slice we have benched, and we have enough plumbing (TRT export, prior-mode framework, multiple checkpoints) to ship something. The remaining "make sure" work is mostly empirical measurement on un-benched slices — about 1 h of work, no retraining, no code changes for items 1–3.